



Ministry of Higher Education
and Scientific Research

ACADEMIC YEAR
2025/2026



Private International Higher
School of Polytechnic

END OF YEAR PROJECT

In order to obtain the National Diploma in Engineering

Field of study: Software Engineering

Entitled

**Printymand: Tunisian Print-on-Demand
Marketplace Platform**

Internship place

Author

Supervisor

Mohamed Amine Zini

Ouassim Tahraoui

Dedication

This project has been made possible thanks to the support and motivation from many individuals to whom we would like to express our deepest gratitude.

We extend our thanks to our beloved parents, who have always been there for us with their support and encouragement.

We also want to express our appreciation to our friends, whose constant presence and motivation have been crucial through this journey.

Thank you all for your support.

Acknowledgments

We would like to express our sincere gratitude to all the professors and academic staff who supported us during this PFA project.

We also thank EPI University for the academic environment and the learning opportunities that helped us develop this project. Finally, we are grateful to our families and friends for their encouragement and support during the project period.

Abstract

Printymand is a web platform designed for print-on-demand commerce. It connects customers, independent designers, printers, and administrators within a unified marketplace workflow. Customers can browse designs, customize products, choose a printer, place orders, and track their fulfillment. Designers can publish artworks, manage product compatibility, and monitor commercial indicators. Printers can manage printable products and update production progress. Administrators can supervise users, content, and global platform activity.

The platform was implemented with Angular and TypeScript for the single-page front-end application, NestJS for the REST back-end API, and PostgreSQL for relational data persistence. The system integrates JWT authentication, role-based authorization, validation, modular services, reusable user interface components, and a structured database model. The adopted methodology is the Spiral Model, which allowed us to develop the project progressively while controlling risks related to security, order integrity, user roles, and marketplace coordination.

This report presents the context, comparative study, requirements, architecture, implementation cycles, and evaluation of Printymand. It shows how the platform answers local print-on-demand needs through a complete academic software engineering project.

Keywords

Print-on-demand, marketplace, Angular, NestJS, PostgreSQL, JWT, role-based access control, Spiral Model, software engineering.

Table of Contents

Dedication	i
Acknowledgments	ii
Abstract	iii
Keywords	iv
List of Figures	viii
List of Tables	x
List of abbreviations	xii
General Introduction	1
1 Preliminary study	3
1.1 Introduction	3
1.2 General overview	3
1.3 Problem	4
1.4 Existing Solutions Study, Criticism, and Proposed Solution	4
1.4.1 Overview of Existing Solutions	4
1.4.1.1 MarYouli	4
1.4.1.2 TeeWinek	4
1.4.1.3 TeePublic	5
1.4.1.4 Redbubble	5
1.4.2 Critical Analysis of Existing Solutions	5
1.4.2.1 MarYouli	5
1.4.2.2 TeeWinek	5
1.4.2.3 TeePublic	6
1.4.2.4 Redbubble	7
1.4.3 Identified Gap	7
1.4.4 Proposed Solution: Printymand	8
1.4.4.1 Key Features	8

1.4.4.2	Strategic Advantage	9
1.5	The adopted methodology	9
1.5.1	The choice of the Spiral Model	10
1.5.2	Why the Spiral Model	10
1.5.3	Project management with the Spiral Model	10
1.5.3.1	Team and roles	12
1.6	Conclusion	12
2	Architecture and planning	13
2.1	Introduction	13
2.2	Requirements specification	13
2.2.1	Identification of actors	13
2.2.2	Functional requirements	14
2.2.3	Non-functional requirements	14
2.3	Global analysis	15
2.3.1	Static diagrams	15
2.3.1.1	Guest use case diagram	15
2.3.1.2	Customer use case diagram	17
2.3.1.3	Designer use case diagram	18
2.3.1.4	Printer use case diagram	19
2.3.1.5	Admin use case diagram	20
2.3.1.6	Global class diagram	21
2.3.2	Dynamic diagrams	22
2.3.2.1	Sequence diagrams	22
2.4	System architecture	26
2.4.1	Front-end architecture	26
2.4.2	Back-end architecture	27
2.4.3	Authentication and authorization	27
2.4.4	Database design	28
2.4.5	Main API modules	28
2.5	Implementation roadmap	29
2.6	Work environment	29
2.6.1	Software architectural pattern	29
2.6.2	Modeling language	29
2.6.3	Hardware environment	30
2.6.4	Software environment	30

2.6.4.1	Development tools	30
2.6.4.2	Libraries, frameworks and programming languages	30
2.7	Conclusion	31
3	Spiral Cycle 1: Authentication, onboarding, and marketplace discovery	32
3.1	Introduction	32
3.2	Cycle planning and objectives	32
3.3	Risk analysis	32
3.3.1	Authentication consistency	32
3.3.2	Role separation	33
3.3.3	Catalog discovery quality	33
3.4	Development	33
3.4.1	Authentication and registration	33
3.4.2	Marketplace and product discovery	34
3.4.3	Reusable components	36
3.4.4	Back-end foundations	36
3.5	Validation and evaluation	36
3.6	Conclusion	36
4	Spiral Cycle 2: Customization, checkout, and order tracking	37
4.1	Introduction	37
4.2	Cycle planning and objectives	37
4.3	Risk analysis	37
4.3.1	Preview-to-production consistency	37
4.3.2	Order integrity	38
4.3.3	Printer reliability	38
4.3.4	Payment workflow	38
4.4	Development	39
4.4.1	Customization interface	39
4.4.2	Printer selection	39
4.4.3	Checkout and payment	39
4.4.4	Order tracking	40
4.5	Validation and evaluation	41
4.6	Conclusion	41
5	Spiral Cycle 3: Governance, administration, and platform optimization	42
5.1	Introduction	42

5.2	Cycle planning and objectives	42
5.3	Risk analysis	42
5.3.1	Governance and moderation	42
5.3.2	Authorization control	43
5.3.3	Operational workflow consistency	43
5.3.4	Maintainability	43
5.4	Development	44
5.4.1	Customer dashboard	44
5.4.2	Designer dashboard	44
5.4.3	Printer dashboard	44
5.4.4	Admin dashboard	45
5.4.5	Back-end optimization and technical services	46
5.4.6	Database consolidation	46
5.5	Validation and evaluation	47
5.6	Conclusion	47
	General Conclusion	48
	Webography	51

List of Figures

1.1	Application logo.	9
1.2	Spiral methodology diagram.	11
2.1	Guest use case diagram.	16
2.2	Customer use case diagram.	17
2.3	Designer use case diagram.	18
2.4	Printer use case diagram.	19
2.5	Admin use case diagram.	20
2.6	Global class diagram.	21
2.7	Customer sequence diagram.	22
2.8	Designer sequence diagram.	23
2.9	Printer sequence diagram.	24
2.10	Admin sequence diagram.	25
2.11	System architecture diagram.	26
2.12	Database domain model diagram.	28
3.1	Screenshot of the login interface.	34
3.2	Screenshot of the registration interface.	34
3.3	Screenshot of the marketplace discovery interface.	35
3.4	Screenshot of the product or design detail interface.	35
4.1	Screenshot of the product customization interface.	39
4.2	Screenshot of the printer selection interface.	40
4.3	Screenshot of the checkout and payment interface.	40
4.4	Screenshot of the order tracking interface.	41
5.1	Customer dashboard interface screenshot.	44
5.2	Designer dashboard interface screenshot.	45
5.3	Printer dashboard interface screenshot.	45

5.4 Admin dashboard interface screenshot. 46

List of Tables

1.1	Roles and responsibilities in the Spiral Model.	11
2.1	Implementation roadmap.	29
2.2	Hardware environment table.	30
2.3	Development tools table.	30
2.4	Libraries, frameworks and programming languages table.	30
3.1	Cycle 1 objectives and deliverables.	33
4.1	Cycle 2 objectives and deliverables.	38
5.1	Cycle 3 objectives and deliverables.	43

List of abbreviations

API	Application Programming Interface
CSS	Cascading Style Sheets
DTO	Data Transfer Object
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
PFA	Projet de Fin d'Annee
POD	Print-On-Demand
REST	Representational State Transfer
RBAC	Role-Based Access Control
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

General Introduction

Digital commerce has progressively moved from fixed catalog sales toward more flexible models where personalization, local fulfillment, and creator participation play a central role. Print-on-demand (POD) is one of these models. Instead of producing large quantities of products in advance, a POD platform prints an item only after a customer order is confirmed. This reduces stock risk, gives designers a channel to monetize digital artwork, and allows printers to receive targeted production requests.

However, a POD marketplace is not only an online store. It must coordinate several actors with different objectives. Customers need to discover designs, preview personalized products, select a suitable printer, pay, and track orders. Designers need a space to publish artworks, manage their profile, and follow sales indicators. Printers need to expose products and production capacity, receive order lines, and update fulfillment status. Administrators need visibility over users, content, orders, and platform health. These requirements create a multi-role software engineering problem involving user experience design, role-based access control, transactional workflows, database modeling, and maintainable full-stack architecture.

In this context, we developed **Printymand**, an academic PFA project carried out as part of the Engineering Degree in Software Engineering at EPI University. The project was developed by two students: Mohamed Amine Zini, responsible for the front-end application, and Ouassim Tahraoui, responsible for the back-end application. Our objective was to build a complete academic platform that demonstrates the main functional and technical aspects of a local print-on-demand marketplace.

Printymand was implemented as an Angular single-page application connected to a NestJS REST API and a PostgreSQL database. The platform includes authentication, marketplace browsing, design and product management, product customization, printer selection, cart and checkout workflow, order tracking, role-specific dashboards, and administration features. These modules were developed to provide a coherent user experience and a

structured technical foundation.

This report is structured as follows:

- The first chapter, **Preliminary Study**, presents the project context, problem statement, comparative study of existing solutions, proposed solution, and adopted Spiral methodology.
- The second chapter, **Architecture and Planning**, details the requirements, actors, global architecture, data model, API structure, work environment, and planning of the Spiral cycles.
- The third chapter, **Spiral Cycle 1**, focuses on authentication, onboarding, marketplace discovery, and the first visible user experience.
- The fourth chapter, **Spiral Cycle 2**, presents product customization, printer selection, cart, checkout, and order tracking.
- The fifth chapter, **Spiral Cycle 3**, describes governance, role dashboards, administration, platform optimization, and maintainability concerns.

Finally, the report ends with a general conclusion summarizing the achievements, lessons learned, and future perspectives of the Printymand platform.

Chapter 1

Preliminary study

1.1 Introduction

This chapter initiates our project study by presenting the domain, motivation, scope, and importance of Printymand. We first introduce the general context of print-on-demand, then identify the problems addressed by our platform. After that, we study representative existing solutions, criticize their strengths and limitations, and present our proposed solution. Finally, we explain the methodology adopted for the project.

1.2 General overview

The project **Printymand** was carried out as part of the PFA for the academic year 2025/2026 at EPI University. It is a Tunisian web-based print-on-demand marketplace. POD allows products to be printed and fulfilled only after an order is placed, which removes the need for large inventories and enables customers to access personalized products.

Our platform connects designers, local printers, suppliers, customers, and administrators in a collaborative ecosystem. Designers can publish artworks and earn royalties. Printers can offer printable products and manage production. Customers can browse designs, customize products, choose a printer, and track orders. Administrators supervise the platform and ensure proper governance.

1.3 Problem

Although print-on-demand removes inventory constraints, Tunisian creators and buyers still face several barriers. International marketplaces frequently exclude Tunisia or impose high shipping costs and long delivery times. Payment and payout systems used by these platforms are often unavailable or impractical for local users. Local providers, on the other hand, generally operate as direct printing shops. They offer basic customization, but they do not provide designer storefronts, royalty mechanisms, advanced dashboards, or a scalable multi-printer ecosystem.

As a result, designers lack local opportunities to monetize their work, customers have limited access to culturally relevant personalized products, and printers cannot benefit from a unified marketplace that generates regular demand. Printymand addresses this problem by proposing a local POD platform adapted to Tunisian users, designers, and printing partners.

1.4 Existing Solutions Study, Criticism, and Proposed Solution

1.4.1 Overview of Existing Solutions

The following short profiles present representative local and international POD providers used in our comparative study.

1.4.1.1 MarYouli

MarYouli is a Tunisian service focused on personalized clothing such as t-shirts and hoodies. It primarily serves local customers who require simple customization and direct purchase.

1.4.1.2 TeeWinek

TeeWinek offers personalized gifts and apparel within Tunisia, with an emphasis on straightforward customization and social-media-driven sales.

1.4.1.3 TeePublic

TeePublic is an international creator marketplace that enables designers to open storefronts and sell designs across a broad product catalog. Fulfillment and shipping are handled by third-party partners.

1.4.1.4 Redbubble

Redbubble is a global marketplace for independent artists, providing an extensive product range and automated order fulfillment through partner networks.

1.4.2 Critical Analysis of Existing Solutions

1.4.2.1 MarYouli

Strengths:

- Local production and delivery enable quick shipping within Tunisia.
- It offers acceptable print quality and customization for personal gifts.
- It operates in the local currency, simplifying payments for Tunisian customers.

Weaknesses:

- No designer marketplace: designers cannot open a storefront or earn royalties.
- Limited product catalog: the service mainly focuses on t-shirts and apparel, with few accessories or lifestyle items.
- Outdated UI/UX: the interface remains basic, with limited interactivity and weak appeal for younger creative users.
- Manual operations: order tracking and customer updates lack automation.
- No analytics or marketing tools: designers and business users cannot monitor visibility or sales performance.
- The service prints products itself, whereas Printymand creates profit opportunities for printers, designers, and suppliers through a collaborative marketplace.

1.4.2.2 TeeWinek

Strengths:

- Localized pricing and delivery make the service accessible for Tunisian customers.
- The customization process is simple for personal designs or slogans.
- The brand is active on social media, which helps build local recognition.

Weaknesses:

- No designer collaboration model: creators cannot publish and sell artwork through personal storefronts.
- Poor UI/UX: the design interface and website layout are not sufficiently engaging, and navigation lacks a professional e-commerce experience.
- Narrow product focus: the catalog is mainly limited to apparel and basic items such as mugs or phone cases.
- Lack of automation: orders and production rely heavily on manual processes.
- Fulfillment capacity constraint: the service operates as an in-house/local printer, limiting its ability to handle large order volumes.
- Like other local providers, TeeWinek prints products itself, while Printymand integrates printers, designers, and suppliers into a shared value chain.

1.4.2.3 TeePublic

Strengths:

- Full creator marketplace: designers can open storefronts, upload artwork, and receive royalties per sale.
- Large product range: the platform offers many items from clothing to accessories and home decor.
- Integrated marketing: products can be promoted through social media or embedded in other stores.
- Automated fulfillment: orders, printing, and shipping are handled seamlessly.

Weaknesses:

- No practical shipping to Tunisia: Tunisian users often cannot place orders or face very high shipping costs.
- Payment barriers: the platform requires payment and payout methods that are not

fully supported in Tunisia.

- Cultural mismatch: artwork and promotional content mainly target Western customers.
- No local pricing: prices are based on foreign currencies, making the service less accessible to Tunisian buyers.

1.4.2.4 Redbubble

Strengths:

- Massive global reach: the platform has millions of customers and active artists worldwide.
- Extensive catalog: it offers a wide range of product types, from clothing to wall art and accessories.
- Automated workflow: printing, shipping, and customer service are managed by the platform.
- Strong brand recognition: it is trusted internationally by artists and buyers.

Weaknesses:

- Tunisia shipping issues: delivery is either unavailable, very slow, or expensive.
- Payment incompatibility: payouts through international services are inaccessible for many Tunisian creators.
- High competition: the large number of designs makes it difficult for new artists to stand out.
- Cultural disconnect: Western-centric aesthetics and pricing reduce relevance for Tunisian consumers.

1.4.3 Identified Gap

From the comparative analysis, several gaps are clear:

- There is no local Tunisian platform that combines local production and shipping with an open designer marketplace, storefronts, and royalty payments.
- Existing local platforms have weaknesses in UI/UX, which reduces their appeal for creatives and younger customers.

- International marketplace platforms are difficult to access for Tunisian customers and designers because of shipping and payout limitations.
- Product diversification remains limited in local solutions, and designers lack dashboards, analytics, and branding control.
- There is no Tunisian platform that integrates printers, suppliers, and designers into a collaborative ecosystem.

1.4.4 Proposed Solution: Printymand

After studying the POD market and the existing solutions, we designed Printymand as a Tunisian web-based POD marketplace focused on local production, local designers, and an integrated multi-role ecosystem.

1.4.4.1 Key Features

Designer Storefronts and Royalties

Designers can register, publish their designs, manage their profile, assign products to their artworks, and follow earnings and performance indicators.

Local Production and Shipping

Products are produced and fulfilled through local printers, which reduces delivery complexity and improves accessibility for Tunisian customers.

Diverse Product Catalog

The platform supports a focused catalog of popular POD items such as t-shirts, hoodies, mugs, stickers, hats, bags, and accessories.

Localized Payments and Payouts

Printymand supports payment methods adapted to Tunisian users and designer payout workflows.

High Quality UI/UX

The interface was designed to be modern, clear, and attractive for customers, designers, and printers. The workflow covers browsing, customization, order placement, and dashboard management.

Collaborative Ecosystem

Printymand integrates printers, designers, customers, and administrators into one platform, generating value for all stakeholders instead of limiting the model to a single printing shop.

1.4.4.2 Strategic Advantage

By focusing on the Tunisian market, Printymand offers faster delivery, lower shipping costs, simplified payments, and better cultural relevance. The designer marketplace model attracts creative talent that lacks local monetization channels, while the printer partnership model increases production capacity and creates a more scalable business structure.



Figure 1.1: Application logo.

1.5 The adopted methodology

Choosing the right methodology is crucial for the success of any software project. It provides an organized way to manage objectives, risks, deadlines, and validation. For Printymand, we adopted the Spiral Model because it combines iterative development with continuous risk analysis.

1.5.1 The choice of the Spiral Model

The Spiral Model is an iterative and risk-driven software development model. Each cycle includes objective definition, risk analysis, engineering work, validation, and planning for the next cycle. This model was suitable for our project because Printymand involves several actors, multiple workflows, and important technical risks such as authentication, authorization, order integrity, payment flow, and data consistency.

1.5.2 Why the Spiral Model

The Spiral Model offered several advantages for our project:

- **Risk management:** we identified security, role separation, order tracking, and marketplace coordination risks early in the project.
- **Flexibility:** each cycle allowed us to refine the platform according to the needs of customers, designers, printers, and administrators.
- **Progressive validation:** every cycle produced functional results that could be evaluated before moving to the next cycle.
- **Controlled complexity:** the project was divided into manageable cycles without losing the global vision of the platform.

1.5.3 Project management with the Spiral Model

We organized the work into three main Spiral cycles:

- **Cycle 1:** authentication, onboarding, marketplace discovery, and reusable interface foundations.
- **Cycle 2:** product customization, printer selection, cart, checkout, payment workflow, and order tracking.
- **Cycle 3:** role dashboards, administration, platform governance, monitoring, and optimization.

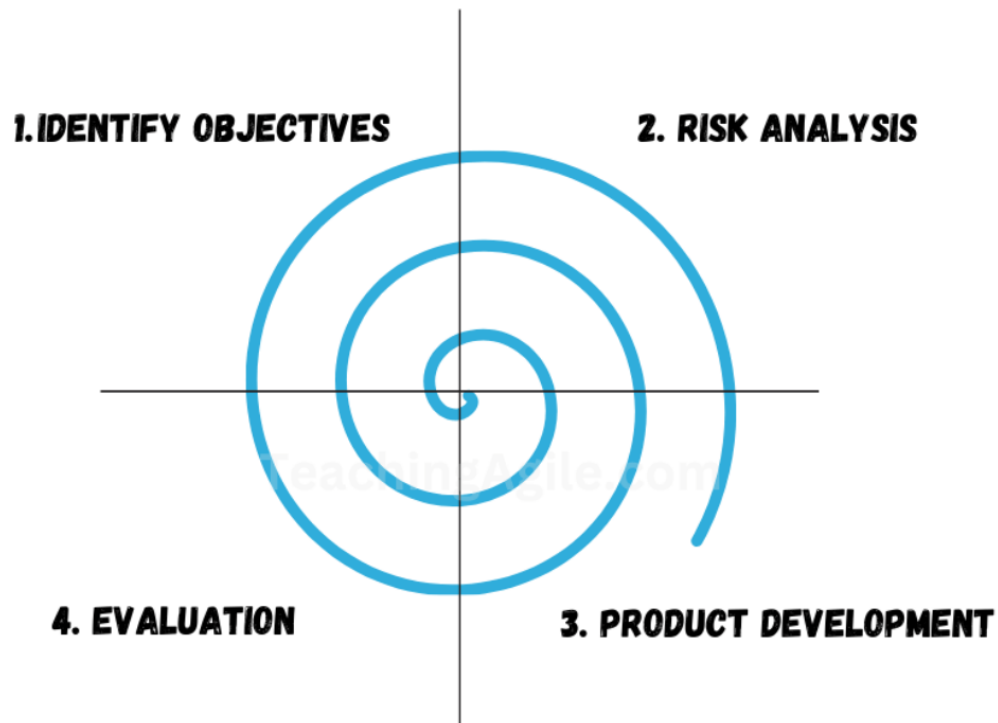


Figure 1.2: Spiral methodology diagram.

Table 1.1: Roles and responsibilities in the Spiral Model.

Role	Mission	Actor
Front-end developer	Design and implementation of the Angular application, user interfaces, services, and role-specific workflows.	Mohamed Amine Zini
Back-end developer	Design and implementation of the NestJS API, authentication, persistence, database schema, guards, and server-side services.	Ouassim Tahraoui

1.5.3.1 Team and roles

1.6 Conclusion

In this chapter, we introduced the project context, analyzed existing POD solutions, identified the main market gaps, and presented Printymand as our proposed solution. We also explained the Spiral Model and how it guided our development. The next chapter presents the architecture, requirements, and planning of the platform.

Chapter 2

Architecture and planning

2.1 Introduction

This chapter presents the technical foundation of Printymand. We define the actors, functional and non-functional requirements, global architecture, database model, front-end and back-end organization, and work environment. We also explain how the development was planned according to the Spiral Model.

2.2 Requirements specification

2.2.1 Identification of actors

An actor represents any external entity that interacts with the system. In Printymand, we identified five main actors:

- **Visitor:** explores the public pages of the platform and can create an account or log in.
- **Customer:** browses the marketplace, customizes products, selects printers, manages the cart, places orders, tracks fulfillment, and submits reviews.
- **Designer:** publishes artworks, manages design information, configures product compatibility, and follows sales and engagement indicators.
- **Printer:** manages printable products, receives order lines, and updates production statuses.
- **Administrator:** supervises users, roles, ranks, content, platform indicators, and

system health.

2.2.2 Functional requirements

Functional requirements describe the actions that the system must perform. The main functional requirements of Printymand are:

- The system must allow users to register and authenticate according to their role.
- The system must protect private pages and API resources through authentication and role-based authorization.
- The system must display a home page, marketplace page, product catalog, and design detail page.
- The system must allow customers to customize a selected design on an available product by changing color, size, position, and scale.
- The system must allow customers to select a printer before adding a customized product to the cart.
- The system must allow cart review, checkout, payment method selection, and order confirmation.
- The system must provide order tracking through a timeline based on production and delivery status.
- The system must allow customers to manage profile information, saved addresses, payment preferences, and reviews.
- The system must allow designers to manage designs, product assignments, profile information, and analytics.
- The system must allow printers to manage printable products and update fulfillment statuses.
- The system must allow administrators to supervise users, content, roles, ranks, and platform activity.
- The system must provide health and monitoring information for administration.

2.2.3 Non-functional requirements

Non-functional requirements define the quality attributes of the system:

- **Security:** authentication is based on JWT, passwords are hashed, and access is controlled through role-based authorization.
- **Usability:** the interface is organized around clear navigation, reusable components, forms, dashboards, and visual feedback.
- **Maintainability:** the system uses a modular architecture separating interface, services, application logic, domain contracts, and infrastructure.
- **Reliability:** validation, exception handling, logging, and transactions contribute to stable behavior.
- **Scalability:** the database model separates users, designs, products, printers, orders, and related entities, which supports marketplace growth.
- **Performance:** pagination, local state management, and request throttling help maintain a responsive user experience.

2.3 Global analysis

2.3.1 Static diagrams

2.3.1.1 Guest use case diagram

The guest use case diagram presents the principal interactions between the unauthenticated user (guest) and the platform.

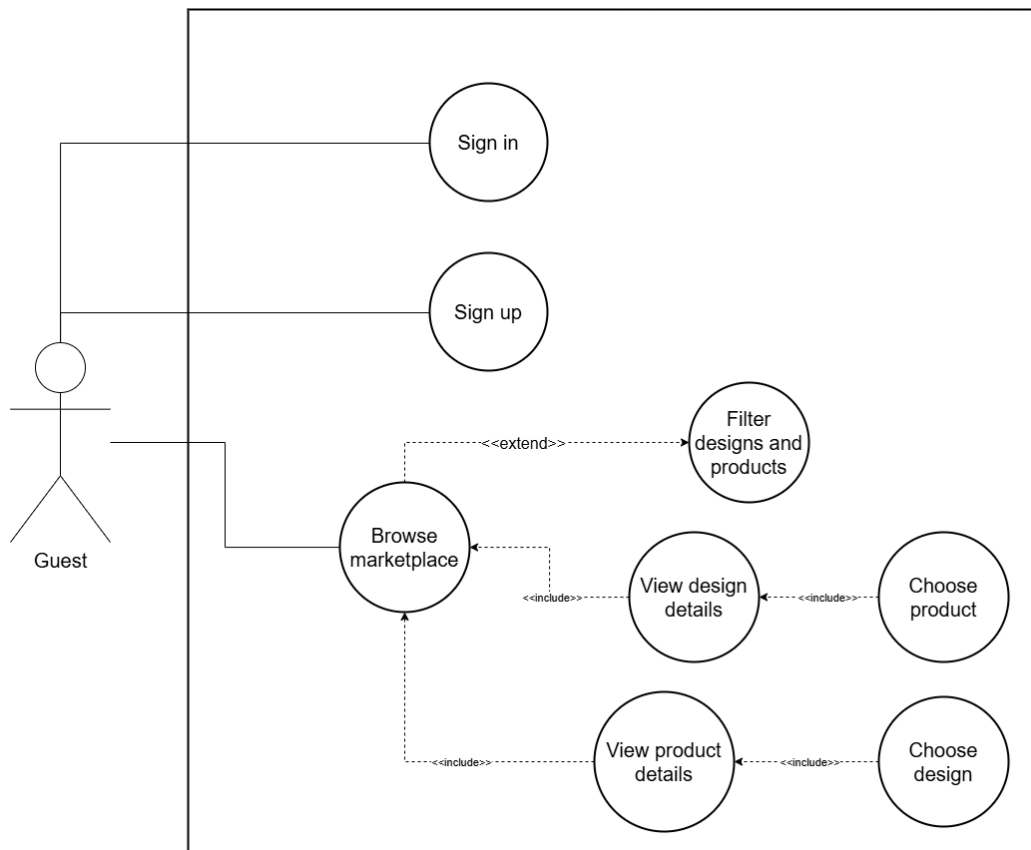


Figure 2.1: Guest use case diagram.

2.3.1.2 Customer use case diagram

The customer use case diagram presents the principal interactions between the customer and the platform.

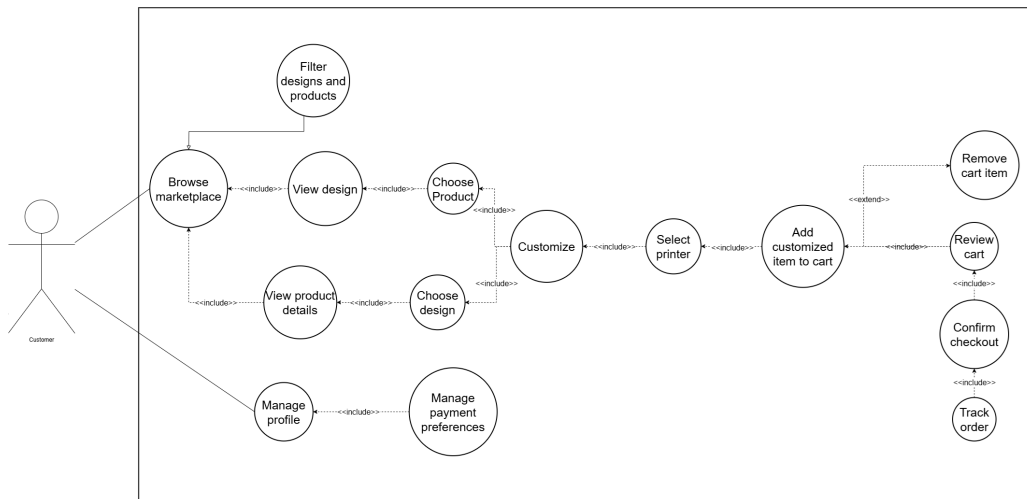


Figure 2.2: Customer use case diagram.

2.3.1.3 Designer use case diagram

The designer use case diagram presents the principal interactions between the designer and the platform.

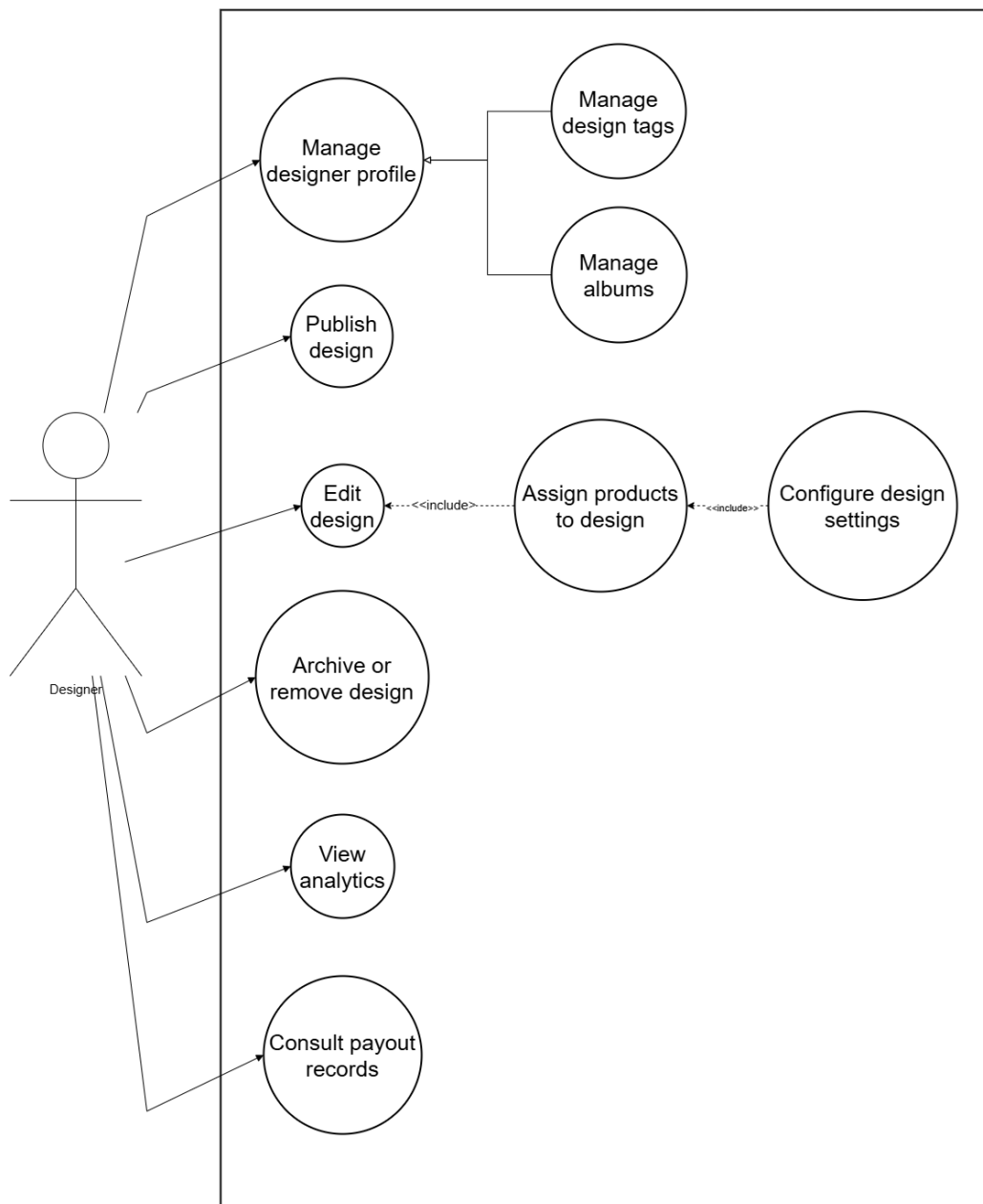


Figure 2.3: Designer use case diagram.

2.3.1.4 Printer use case diagram

The printer use case diagram presents the principal interactions between the printer and the platform.

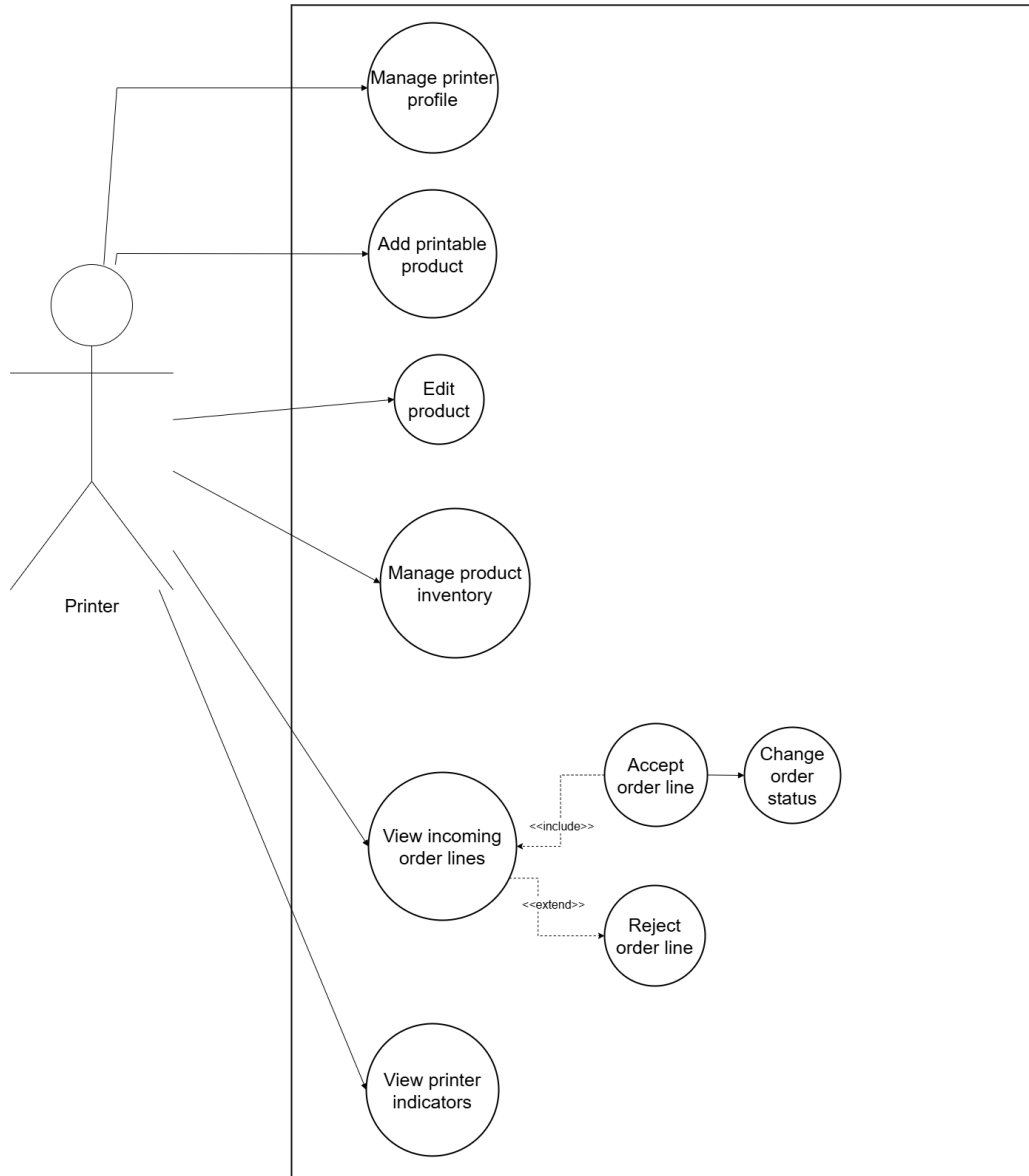


Figure 2.4: Printer use case diagram.

2.3.1.5 Admin use case diagram

The admin use case diagram presents the principal interactions between the admin and the platform.

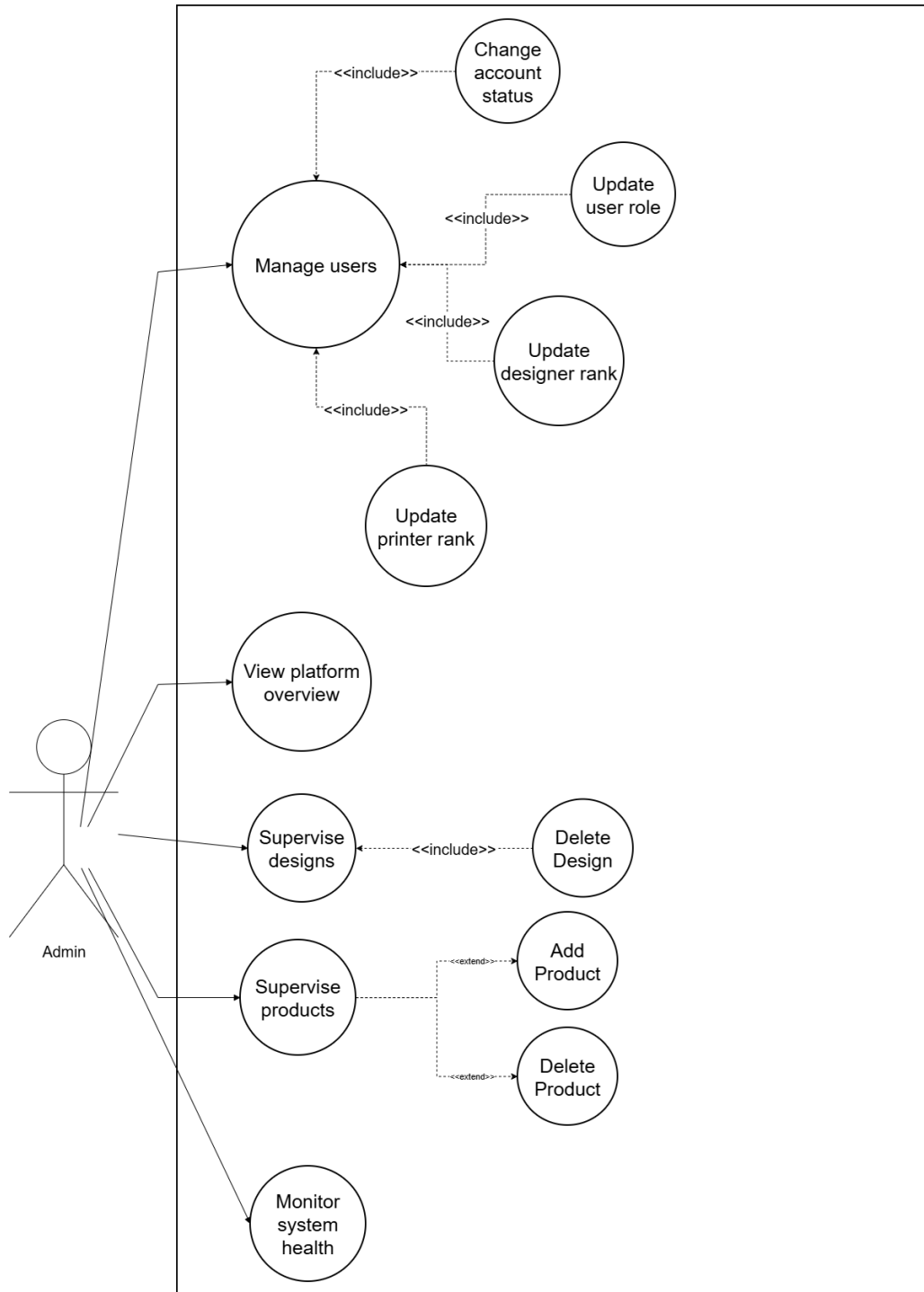


Figure 2.5: Admin use case diagram.

2.3.1.6 Global class diagram

The global class diagram represents the main domain concepts of Printymand. The principal entities are User, CustomerProfile, DesignerProfile, PrinterProfile, Design, Product, PrinterPartner, CustomizationDraft, CartItem, Order, OrderLine, Review, and PayoutRecord. These classes model the marketplace workflow from publication to order fulfillment.

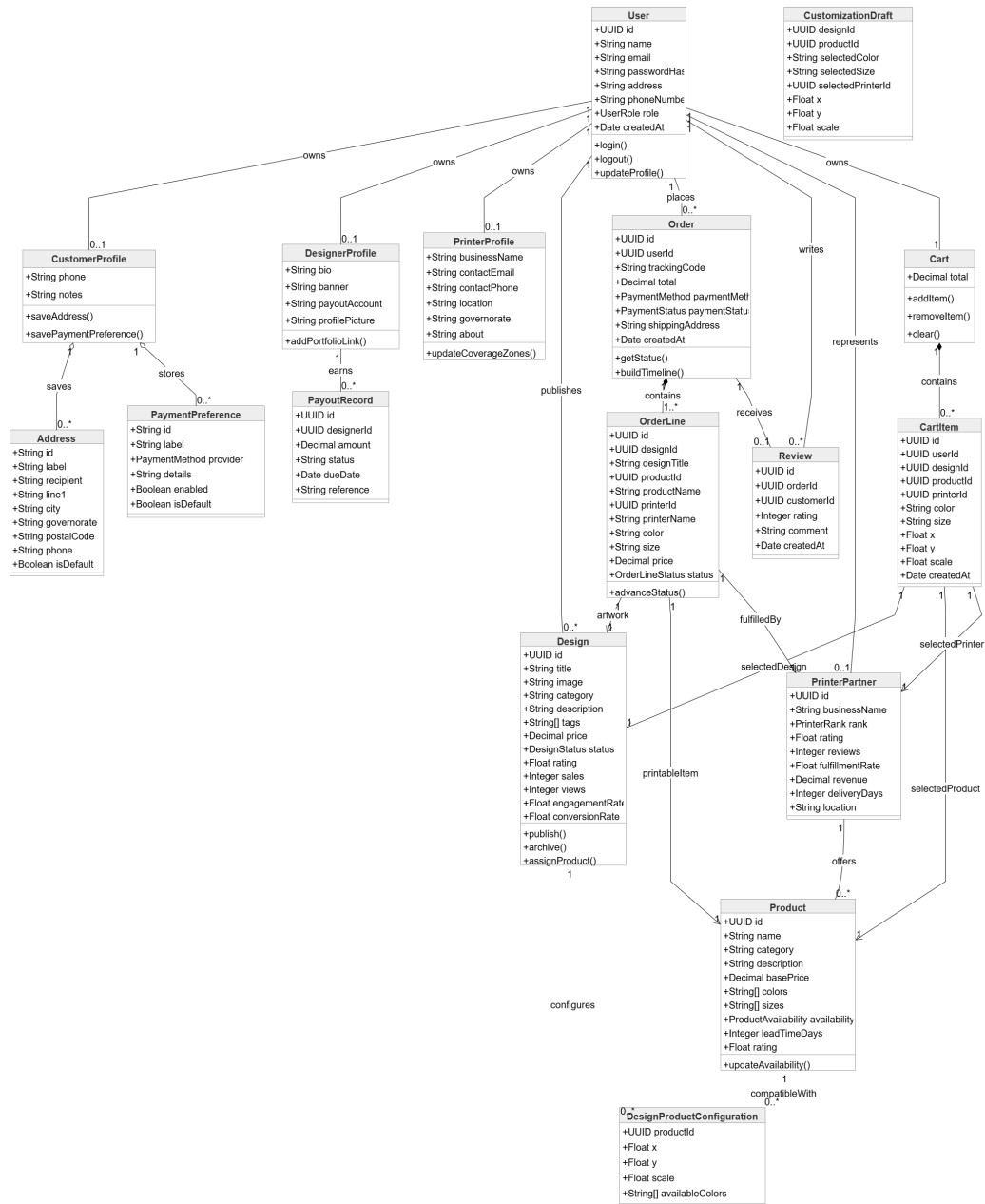


Figure 2.6: Global class diagram.

2.3.2 Dynamic diagrams

2.3.2.1 Sequence diagrams

The following sequence diagrams show the chronological flow of the journey of a customer, designer, printer and an admin respectively. A customer authenticates, browses designs, customizes a product, selects a printer, adds the item to the cart, confirms the order, and follows the tracking timeline. Designers, printers, and administrators interact with their dashboards to complete the operational workflow.

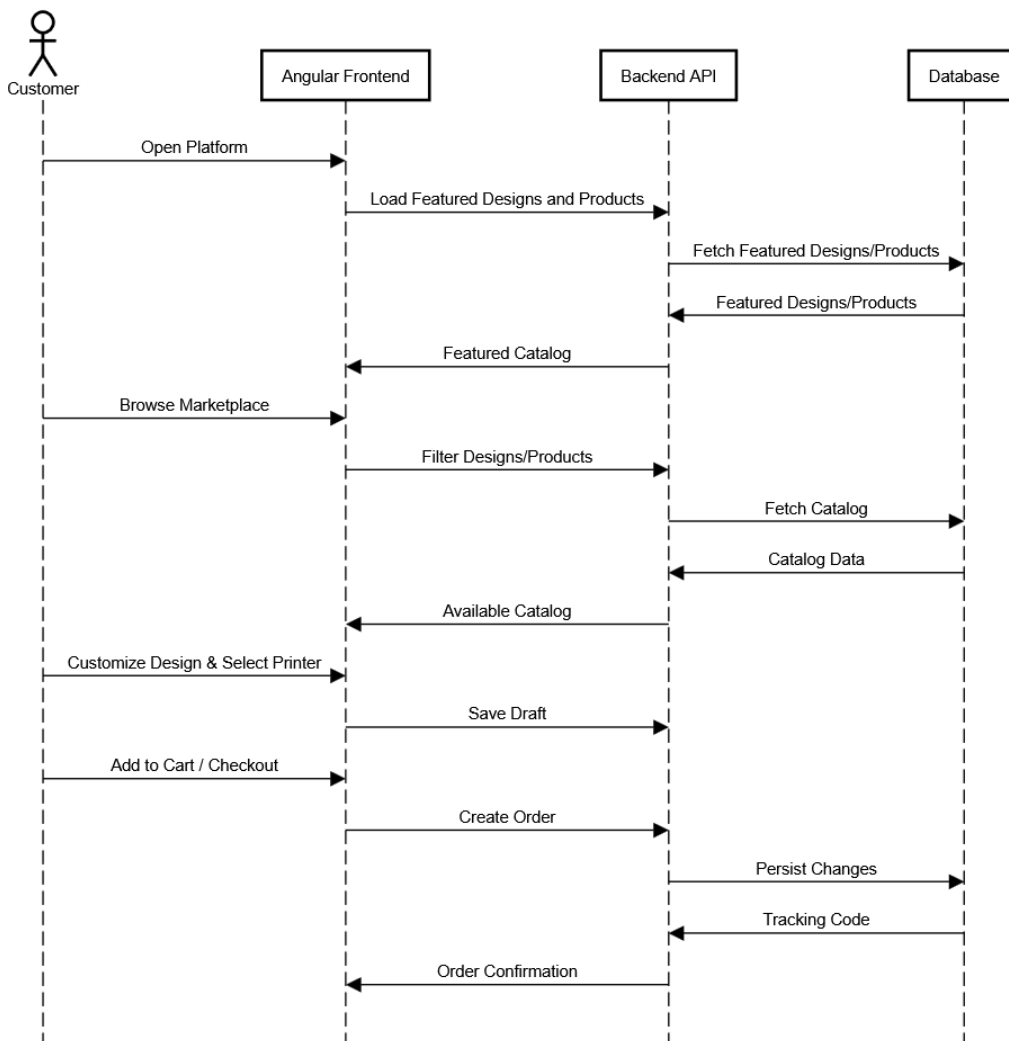


Figure 2.7: Customer sequence diagram.

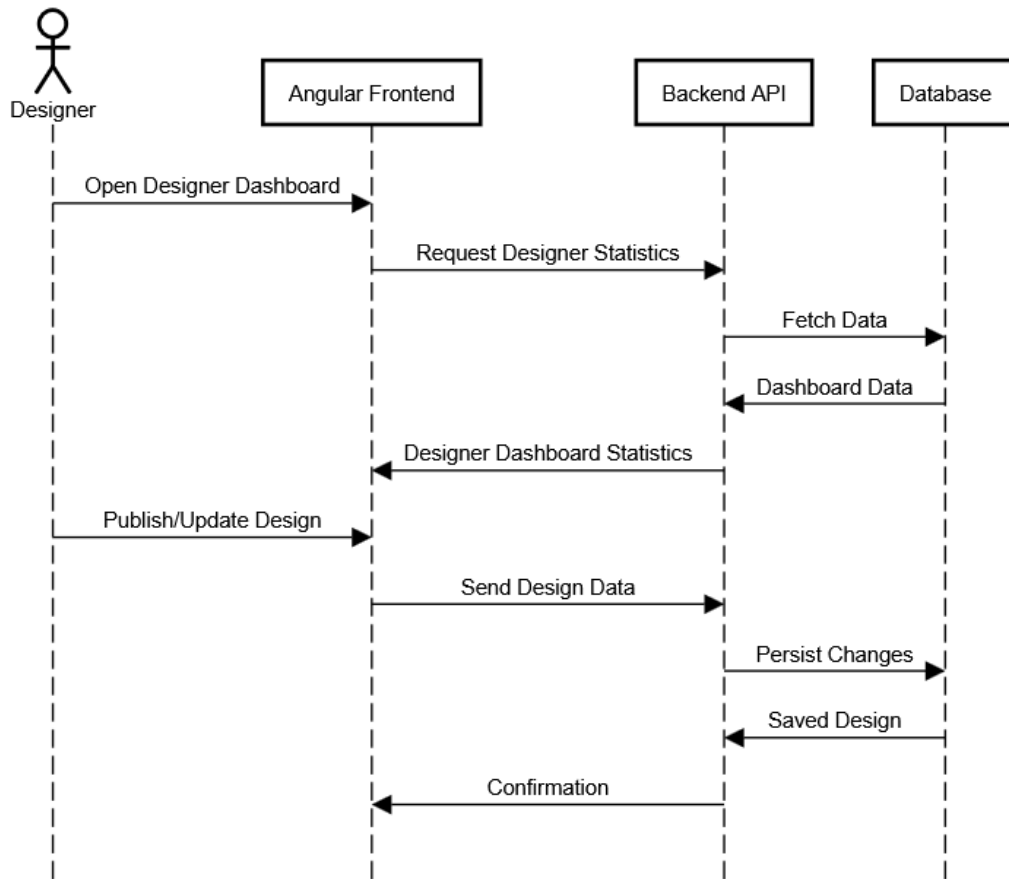


Figure 2.8: Designer sequence diagram.

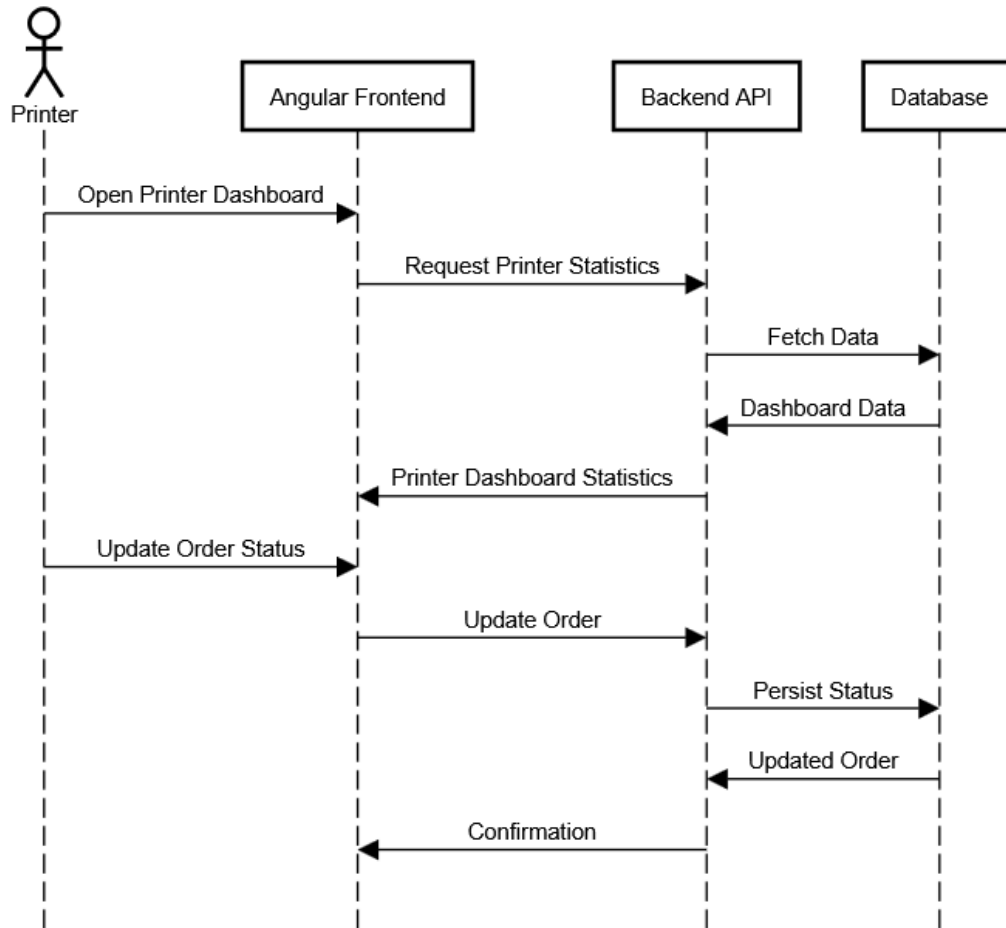


Figure 2.9: Printer sequence diagram.

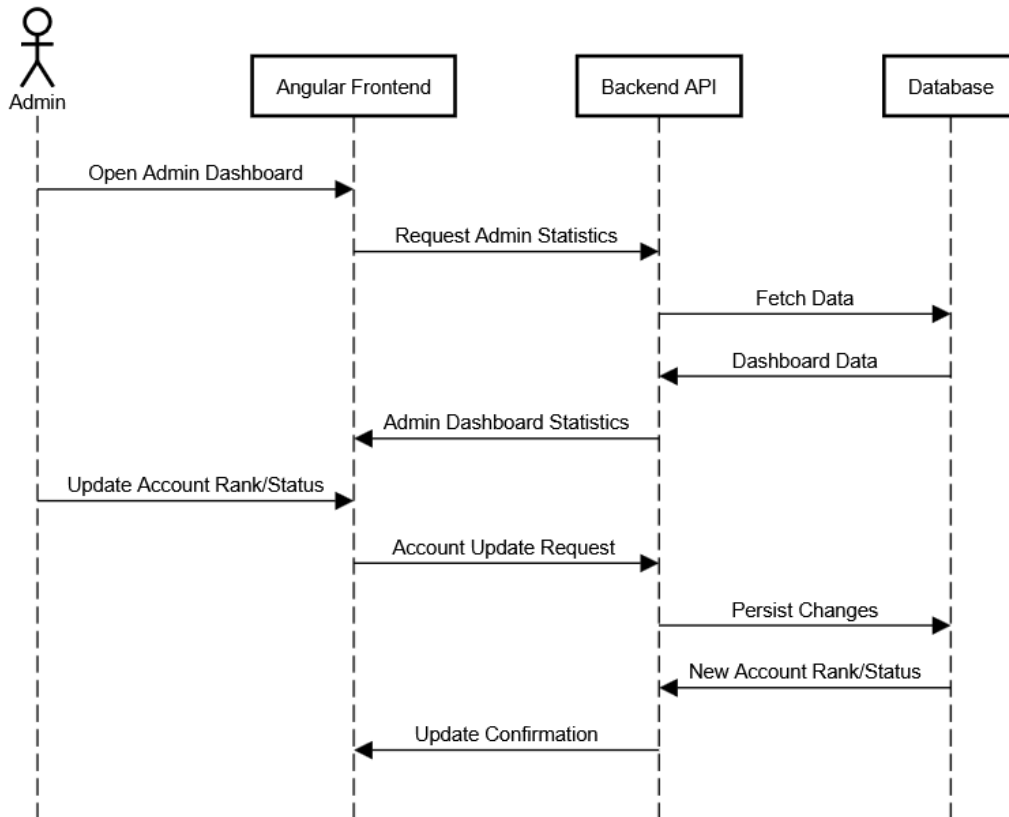


Figure 2.10: Admin sequence diagram.

2.4 System architecture

Printymand follows a client-server architecture. The client side is a single-page Angular application responsible for interaction, navigation, state management, and user interface rendering. The server side is a NestJS REST API responsible for authentication, authorization, business operations, validation, persistence, and monitoring. PostgreSQL is used as the relational database.

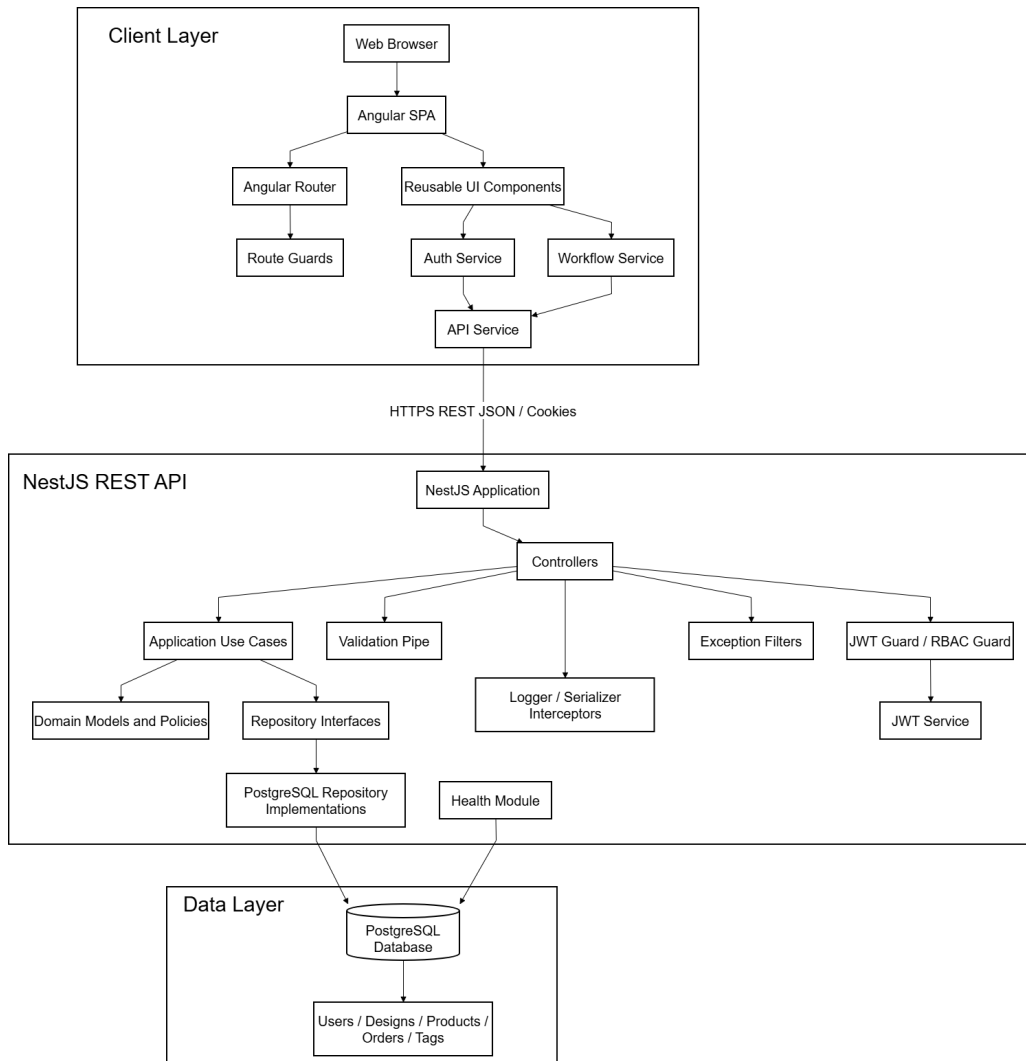


Figure 2.11: System architecture diagram.

2.4.1 Front-end architecture

The front-end application was implemented with Angular and TypeScript. We adopted a component-based architecture to make the interface reusable and maintainable. Pages

represent complete views such as home, marketplace, products, design details, customization, printer selection, checkout, tracking, and dashboards. Components such as design cards, product cards, printer cards, stat cards, rank badges, modals, dashboard shells, and image fallback elements are reused across several pages.

Services centralize application logic. The authentication service manages user access, role redirection, logout, and profile updates. The workflow service coordinates designs, products, printers, cart items, orders, reviews, payouts, and dashboard indicators. The API service communicates with the REST back end. This separation makes the front end easier to maintain and allows each page to focus on presentation and user interaction.

2.4.2 Back-end architecture

The back-end application was implemented with NestJS and TypeScript. We adopted a layered architecture composed of:

- **API layer:** controllers, guards, decorators, filters, interceptors, and request DTOs.
- **Application layer:** use cases that implement business operations such as login, registration, product creation, design publication, dashboard retrieval, and session management.
- **Domain layer:** entities, DTOs, repository contracts, policies, and value objects.
- **Infrastructure layer:** JWT services, PostgreSQL persistence, transactions, health checks, and realtime services.

This architecture separates business logic from technical details and improves testability, readability, and extensibility.

2.4.3 Authentication and authorization

Authentication is handled through JWT tokens. During registration or login, the server verifies the user information, signs a token, and sends it to the client through an HTTP-only cookie. Passwords are hashed with bcrypt. The platform also supports token extraction from authorization headers for API usage.

Authorization is based on RBAC. Guards verify that a user is authenticated and has the required role before accessing protected resources. The same principle is applied in the Angular application through route guards that redirect users to the correct dashboard according to their role.

2.4.4 Database design

The PostgreSQL database was designed around the main marketplace entities:

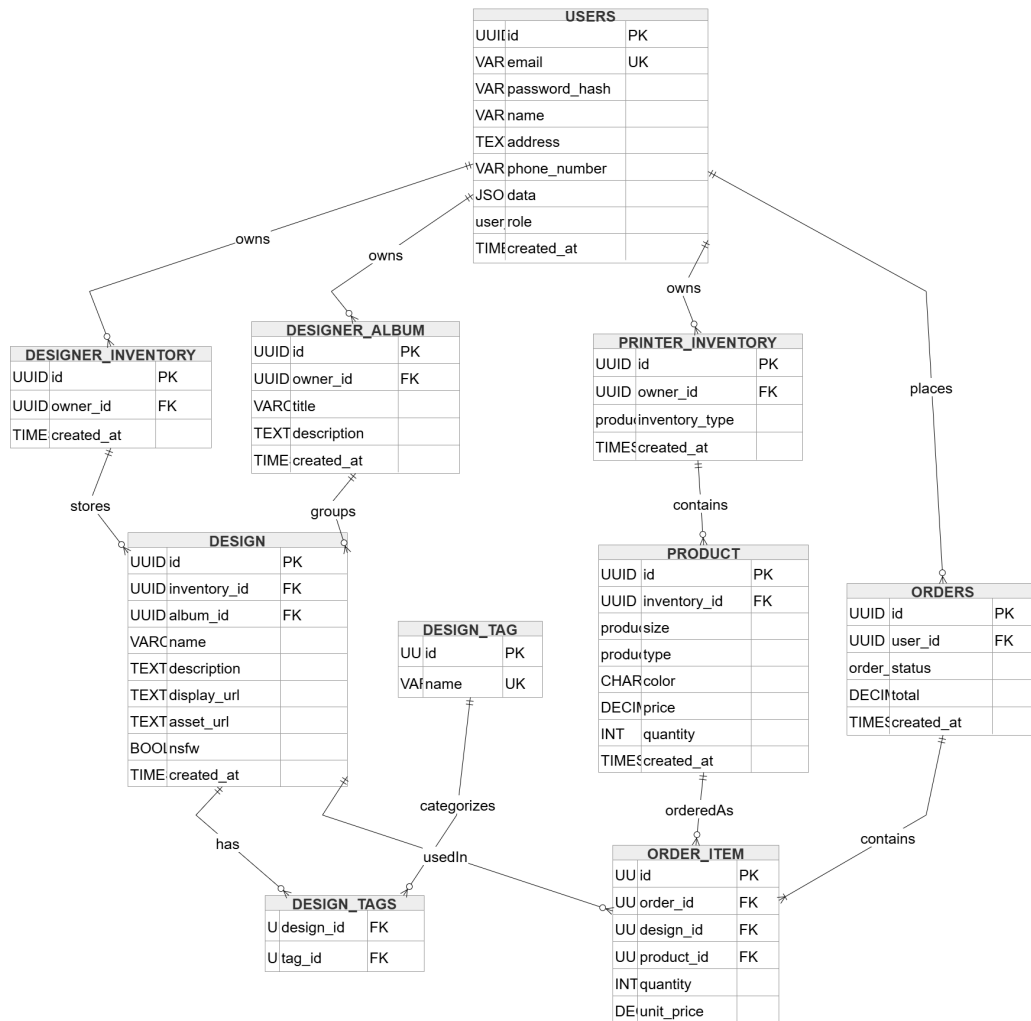


Figure 2.12: Database domain model diagram.

The use of relational tables provides data integrity and supports the relations between customers, designers, printers, products, designs, and orders.

2.4.5 Main API modules

The REST API is organized into modules that reflect the platform domains:

- **Authentication:** registration, login, logout, refresh, and current user retrieval.
- **Products:** product creation, listing, and details.
- **Designs:** design publication, listing, upload configuration, and details.

- **Dashboard:** global and role-related indicators.
- **Cart and orders:** cart workflow, order confirmation, and tracking operations.
- **Health:** administrator monitoring of memory and database connectivity.

2.5 Implementation roadmap

Table 2.1: Implementation roadmap.

Cycle	Focus	Main deliverables
1	Authentication and discovery	Registration, login, role guards, marketplace browsing, product and design pages, reusable interface components
2	Customization and checkout	Customization draft, product preview, printer selection, cart, order placement, payment workflow, and tracking
3	Governance and optimization	Customer, designer, printer, and admin dashboards; PostgreSQL schema; health checks; validation; RBAC; back-end modularization

2.6 Work environment

2.6.1 Software architectural pattern

For the front end, we used a component-service architecture. Components define interface elements and views, while services centralize authentication, workflow, and API communication. For the back end, we used a layered architecture inspired by clean architecture principles, separating controllers, use cases, domain contracts, and infrastructure.

2.6.2 Modeling language

We used UML to model the system because it provides a clear and standardized way to represent actors, use cases, classes, and interactions. UML diagrams helped us understand the system before implementation and communicate design decisions more effectively.

2.6.3 Hardware environment

Table 2.2: Hardware environment table.

Element	Specification
Development machines	Student laptops used for front-end and back-end development
Operating system	Windows development environment
Memory	Suitable for Angular, NestJS, PostgreSQL, and LaTeX authoring
Network	Local development environment for client-server testing

2.6.4 Software environment

2.6.4.1 Development tools

Table 2.3: Development tools table.

Tool	Usage in the project
Visual Studio Code	Source code editing and project navigation.
Git and GitHub	Version control and collaboration.
Node.js and npm	Package management and execution of Angular and NestJS scripts.
PostgreSQL	Relational database used by the back-end persistence layer.
LaTeX	Professional report writing and academic document generation.

2.6.4.2 Libraries, frameworks and programming languages

Table 2.4: Libraries, frameworks and programming languages table.

Technology	Usage in the project
Angular	Framework used to develop the single-page front-end application, routing, components, forms, and services [1].
TypeScript	Main programming language used for both client-side and server-side development [2].

Technology	Usage in the project
NestJS	Framework used to implement the REST API, modules, controllers, guards, providers, and dependency injection [3].
PostgreSQL	Relational database used for persistent platform data [4].
JWT	Token format used for secure authentication [5].
bcrypt	Library used for password hashing and verification.
Tailwind CSS	Styling technology used to support modern and responsive interface design [6].

2.7 Conclusion

In this chapter, we presented the requirements, actors, architecture, database model, API modules, and work environment of Printymand. We also showed how the platform was organized into three Spiral cycles. The next chapter presents the first cycle, which focuses on authentication, onboarding, and marketplace discovery.

Chapter 3

Spiral Cycle 1: Authentication, onboarding, and marketplace discovery

3.1 Introduction

The first Spiral cycle established the foundations of Printymand. Its objective was to make the platform accessible, navigable, and clear for the main user roles. In this cycle, we implemented authentication, registration, role-based redirection, public marketplace discovery, product and design browsing, and reusable user interface components.

3.2 Cycle planning and objectives

3.3 Risk analysis

3.3.1 Authentication consistency

Authentication is a sensitive part of the platform because it protects user data and role-specific dashboards. To reduce this risk, we used JWT authentication, HTTP-only cookies, password hashing, and guarded routes. The authentication service also centralizes login, registration, logout, and profile state, which avoids duplicating access logic across pages.

Table 3.1: Cycle 1 objectives and deliverables.

Objective	Deliverable	Priority
Account access	Login, registration, logout, current user state, and authenticated sessions.	High
Role onboarding	Selection of customer, designer, or printer role during registration.	High
Marketplace discovery	Home page, marketplace page, product catalog, design cards, product cards, and design detail page.	High
Navigation protection	Front-end route guards and back-end JWT/RBAC guards.	High
Reusable UI	Layout, dashboard shell, cards, modal, stat cards, skeleton blocks, rank badges, and image fallback component.	Medium

3.3.2 Role separation

Printymand includes four authenticated roles: customer, designer, printer, and administrator. Each role has specific rights and navigation paths. We addressed this risk through role-based guards on the client side and RBAC guards on the server side. As a result, each user accesses the workspace and operations that correspond to his or her responsibilities.

3.3.3 Catalog discovery quality

Marketplace discovery depends on the clarity of design cards, product cards, categories, and detail pages. We focused on building a clean browsing experience so that customers can understand the offer before starting customization. This choice improves usability and creates a natural transition toward customization.

3.4 Development

3.4.1 Authentication and registration

We implemented authentication around a clear access workflow. During registration, the user provides identity information, address, password, and role. The platform creates the account and initializes the default profile corresponding to the chosen role. During login, the server verifies the credentials, signs a JWT token, and establishes the session.

The authentication interface was designed to be simple and direct. It helps users access

the platform quickly and supports role-based redirection after successful authentication.

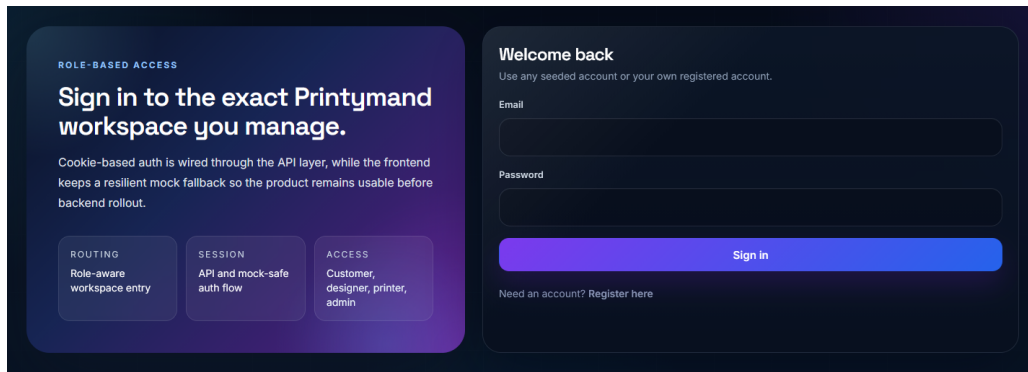


Figure 3.1: Screenshot of the login interface.

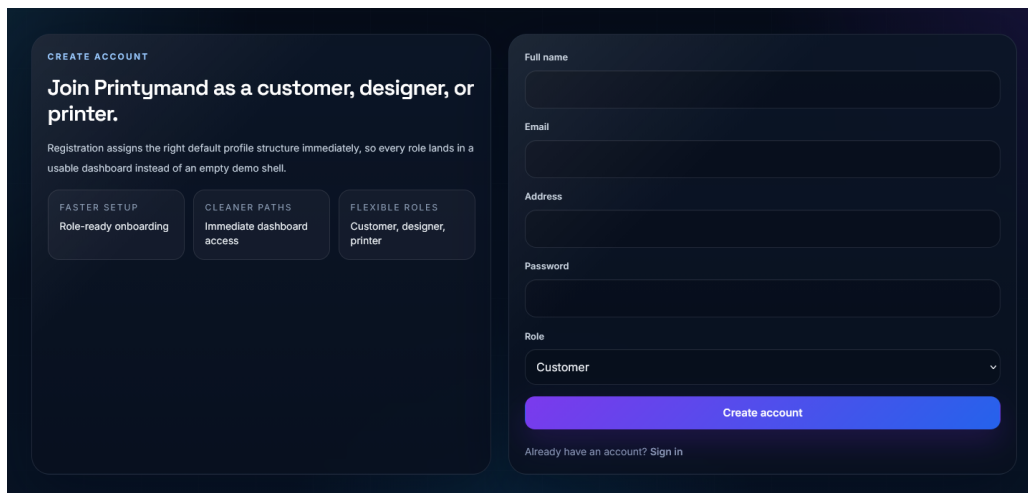


Figure 3.2: Screenshot of the registration interface.

3.4.2 Marketplace and product discovery

The home page introduces the platform and highlights featured designs and products. The marketplace page allows customers to explore available designs, while the product page presents printable items. Each design and product is displayed through reusable cards that include the essential information needed for decision-making.

The design detail page gives the customer more information about the selected artwork and provides access to customization. This flow creates a natural transition from discovery to purchase.

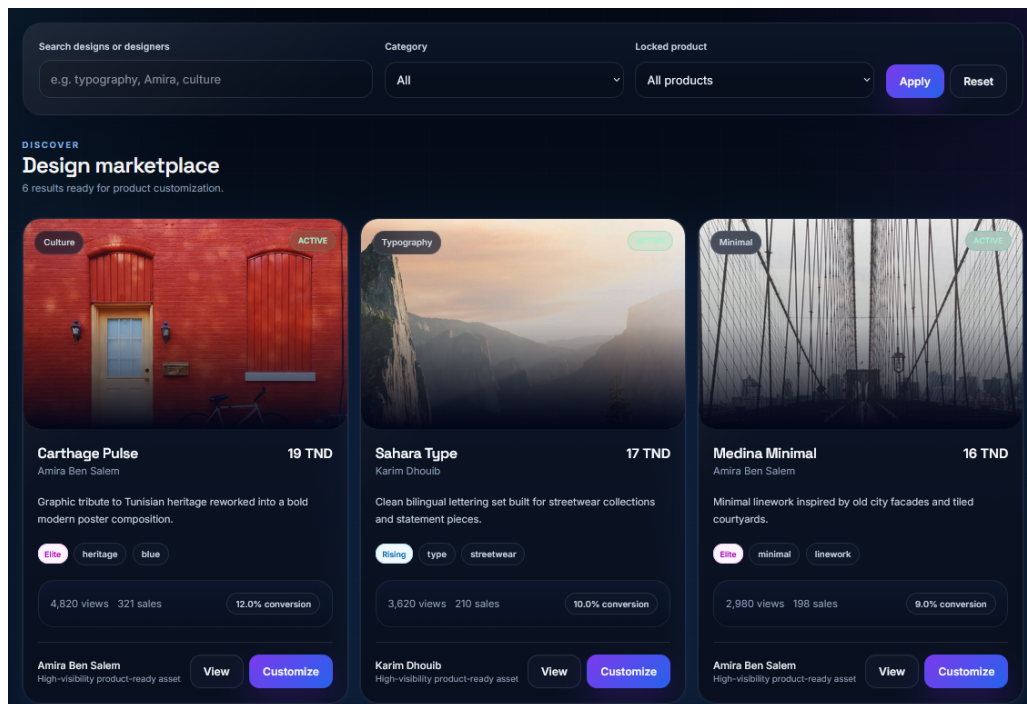


Figure 3.3: Screenshot of the marketplace discovery interface.

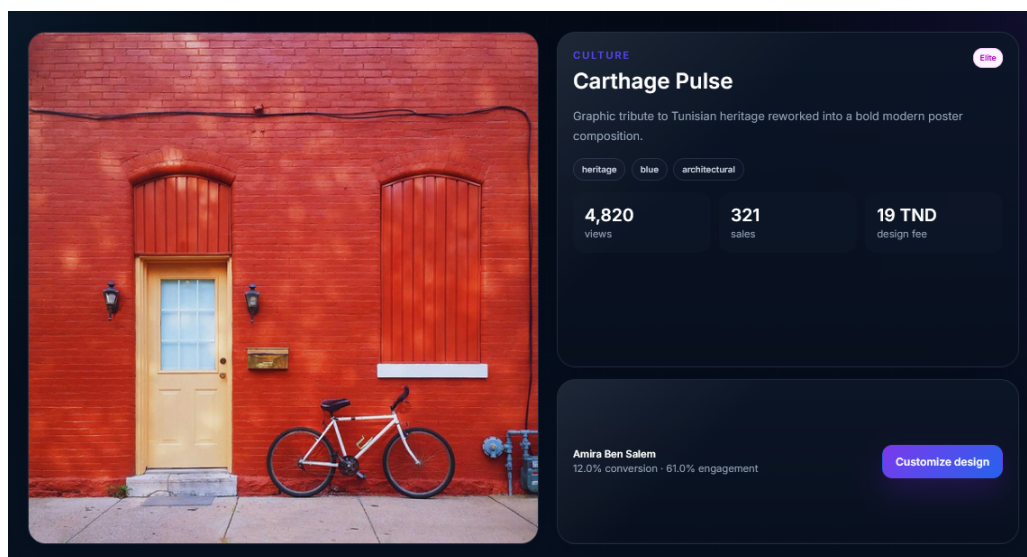


Figure 3.4: Screenshot of the product or design detail interface.

3.4.3 Reusable components

We developed reusable interface components to keep the platform consistent. The layout component structures the main navigation. The dashboard shell supports role-specific workspaces. Design cards, product cards, printer cards, stat cards, rank badges, image fallback, modal, and skeleton blocks reduce repetition and improve maintainability.

This component-oriented approach allowed us to build several pages with a coherent visual identity and stable behavior.

3.4.4 Back-end foundations

The first cycle also established the essential server-side foundations. We configured the NestJS application, API prefix, CORS, cookie parsing, validation, exception filters, logging interceptor, authentication module, JWT utility, and guards. These elements form the technical base used by the rest of the platform.

3.5 Validation and evaluation

At the end of the first cycle, users could register, log in, access the platform according to their role, browse the marketplace, view products and designs, and navigate through a consistent interface. This cycle validated the platform's identity, access management, and discovery workflow.

3.6 Conclusion

The first Spiral cycle established the foundation of Printymand: authentication, onboarding, discovery, navigation, reusable components, and back-end security infrastructure. The next cycle builds on this foundation by implementing customization, printer selection, checkout, and order tracking.

Chapter 4

Spiral Cycle 2: Customization, checkout, and order tracking

4.1 Introduction

The second Spiral cycle focused on the central print-on-demand transaction. After discovering a design, a customer must be able to configure it on a product, select a printer, add the result to the cart, place an order, and track fulfillment. This cycle transformed Printymand from a browsing platform into a complete marketplace workflow.

4.2 Cycle planning and objectives

4.3 Risk analysis

4.3.1 Preview-to-production consistency

In print-on-demand systems, a major risk is the difference between the customer's visual preview and the printable result. We reduced this risk by storing the design position and scale as structured placement values. This allows the platform to preserve customization choices throughout the cart, checkout, and order workflow.

Table 4.1: Cycle 2 objectives and deliverables.

Objective	Deliverable	Priority
Product customization	Design placement, scaling, color selection, size selection, and product preview.	High
Printer selection	Selection of a printer partner before cart insertion.	High
Cart workflow	Draft conversion to cart item, cart line display, total calculation, and item removal.	High
Checkout	Shipping address selection, payment method selection, order finalization, and payment status management.	High
Order tracking	Tracking code, order timeline, order status aggregation, and customer review submission.	Medium

4.3.2 Order integrity

An order depends on several elements: customer, design, product, printer, selected color, selected size, placement, price, shipping address, and payment method. We designed the workflow so that each cart line contains all necessary information before order confirmation. This ensures that the confirmed order remains coherent and traceable.

4.3.3 Printer reliability

Printer selection is essential because fulfillment depends on the selected printing partner. The platform displays printer information such as business name, rank, rating, reviews, fulfillment rate, delivery days, and location. These criteria help customers choose a suitable printer and help printers build visibility.

4.3.4 Payment workflow

The payment workflow supports several payment methods, including cash and digital options. We designed it so that order status and payment status are clearly separated, which makes it easier to track both commercial confirmation and production progress.

4.4 Development

4.4.1 Customization interface

The customization interface allows the customer to select a compatible product, choose color and size, move the design on the product preview, and adjust its scale. The position and scale are saved as a customization draft, which preserves the customer's choices during the transition to printer selection and cart insertion.

This interface is one of the most important parts of the customer experience because it gives visual control over the final product.

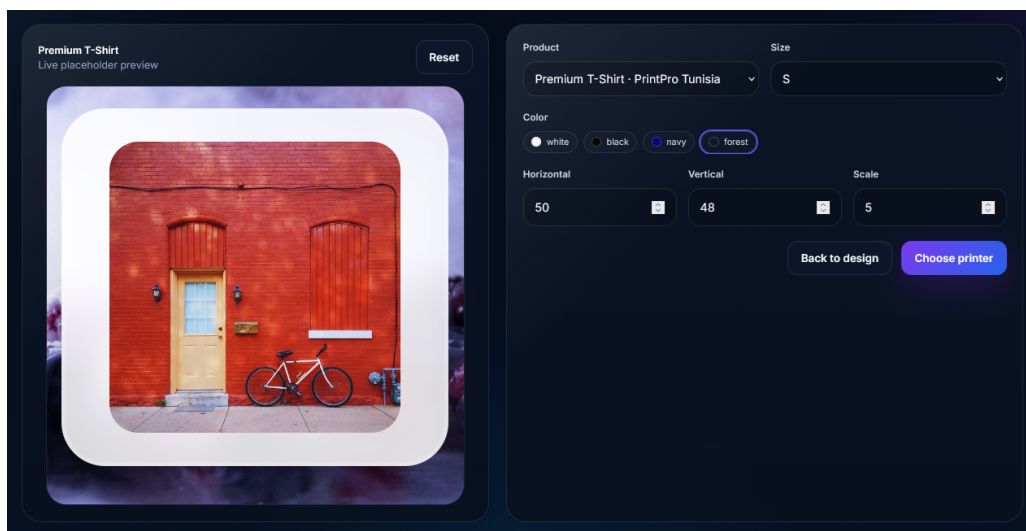


Figure 4.1: Screenshot of the product customization interface.

4.4.2 Printer selection

After customization, the customer selects a printer partner. The printer selection interface presents production-related information that helps the customer compare options. Once a printer is selected, the customized product can be added to the cart with all required details.

4.4.3 Checkout and payment

The checkout page gathers the shipping address and payment method. Once confirmed, the platform creates an order, generates order lines, assigns a tracking code, calculates the

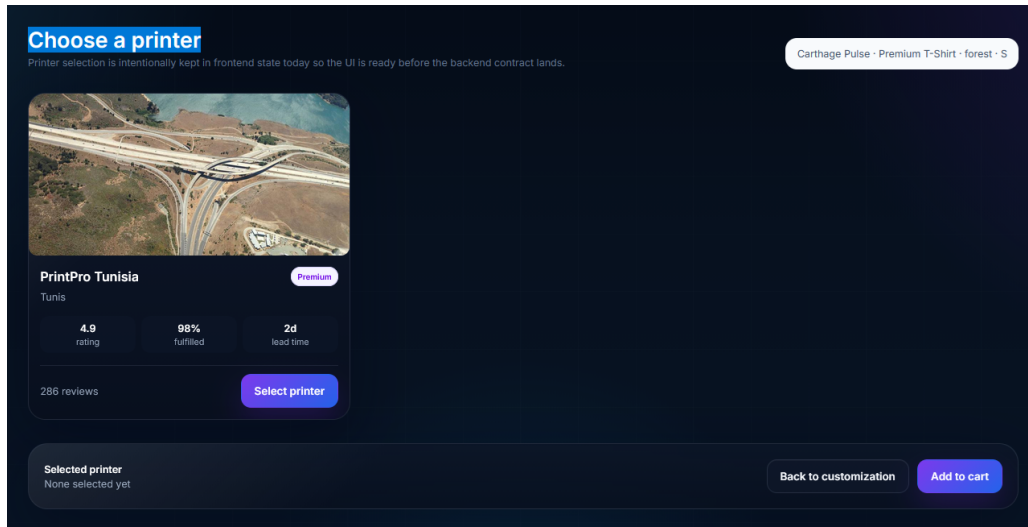


Figure 4.2: Screenshot of the printer selection interface.

total amount, records the payment status, and clears the cart. The user is then redirected to the tracking page.

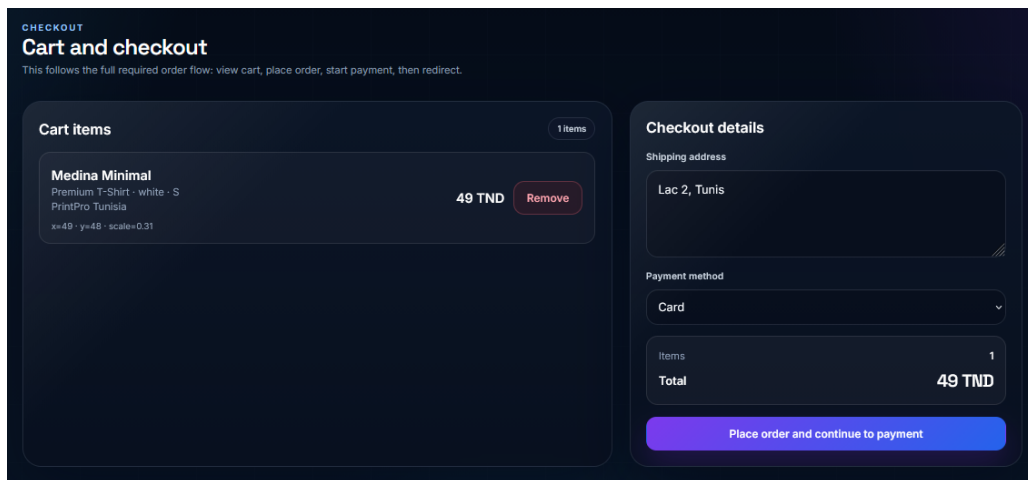


Figure 4.3: Screenshot of the checkout and payment interface.

4.4.4 Order tracking

Order tracking is based on the status of order lines. The timeline includes pending, accepted, printing, shipped, and delivered. This gives customers a clear view of production and delivery progress. Customers can also submit reviews after completing an order, which improves marketplace trust.

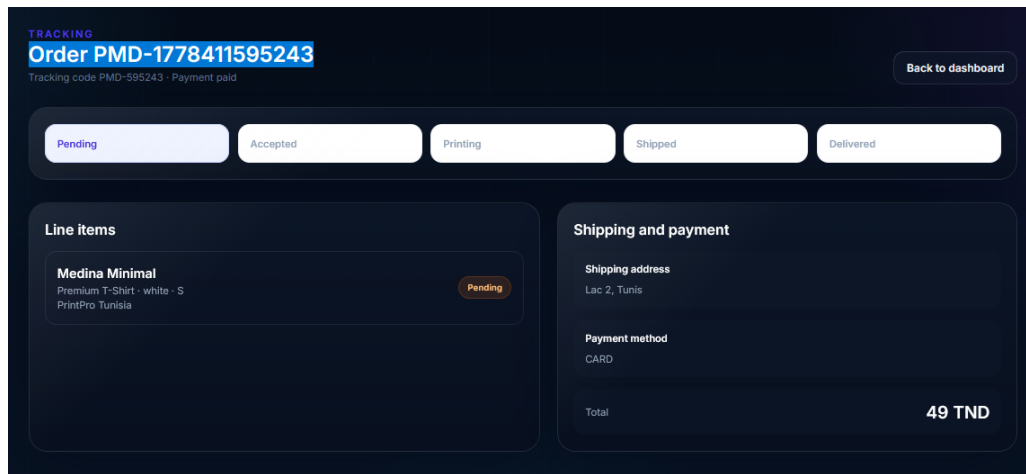


Figure 4.4: Screenshot of the order tracking interface.

4.5 Validation and evaluation

At the end of Cycle 2, the customer workflow was complete: selecting a design, customizing a product, choosing a printer, adding the product to the cart, confirming checkout, and tracking the order. This cycle validated the central commercial process of Printymand and confirmed that the marketplace could coordinate customers, designs, products, printers, and orders in a single coherent workflow.

4.6 Conclusion

The second Spiral cycle implemented the transactional heart of Printymand. It connected marketplace discovery to customization, printer selection, checkout, and tracking. The next cycle focuses on dashboards, administration, governance, and platform optimization.

Chapter 5

Spiral Cycle 3: Governance, administration, and platform optimization

5.1 Introduction

The third Spiral cycle consolidated the platform around role-specific dashboards, governance, administration, and technical optimization. At this stage, Printymand had to provide not only customer ordering features, but also the tools required by designers, printers, and administrators to manage their responsibilities.

5.2 Cycle planning and objectives

5.3 Risk analysis

5.3.1 Governance and moderation

Marketplace platforms require governance because users publish content, printers fulfill orders, and customers depend on reliable operations. We addressed this risk by implementing an administration space where platform activity, users, roles, ranks, and content status can be supervised.

Table 5.1: Cycle 3 objectives and deliverables.

Objective	Deliverable	Priority
Customer workspace	Profile, saved addresses, payment preferences, orders, reviews, and dashboard indicators.	Medium
Designer workspace	Design management, product assignment, analytics, profile, and payout records.	High
Printer workspace	Product management, order-line management, fulfillment status updates, and profile.	High
Administration	User management, role/rank updates, account suspension, content supervision, and platform overview.	High
Technical optimization	Health checks, throttling, validation, filters, logging, transactions, and database schema consolidation.	High

5.3.2 Authorization control

As the number of features increases, access control becomes more important. We applied role-based authorization consistently so that customer, designer, printer, and administrator operations remain separated. This protects sensitive actions such as rank changes, account suspension, health checks, and fulfillment updates.

5.3.3 Operational workflow consistency

Designer analytics, printer fulfillment, customer tracking, and administrator metrics all depend on consistent data. We designed the workflow so that each user action updates the relevant indicators and preserves the relation between users, designs, products, printers, and orders.

5.3.4 Maintainability

The platform was organized with reusable components, centralized services, modular API layers, repository contracts, and structured database migrations. This organization reduced complexity and made the system easier to extend.

5.4 Development

5.4.1 Customer dashboard

The customer dashboard allows users to view their orders, manage profile data, save addresses, define payment preferences, and consult submitted reviews. This workspace gives customers a clear view of their activity on the platform.

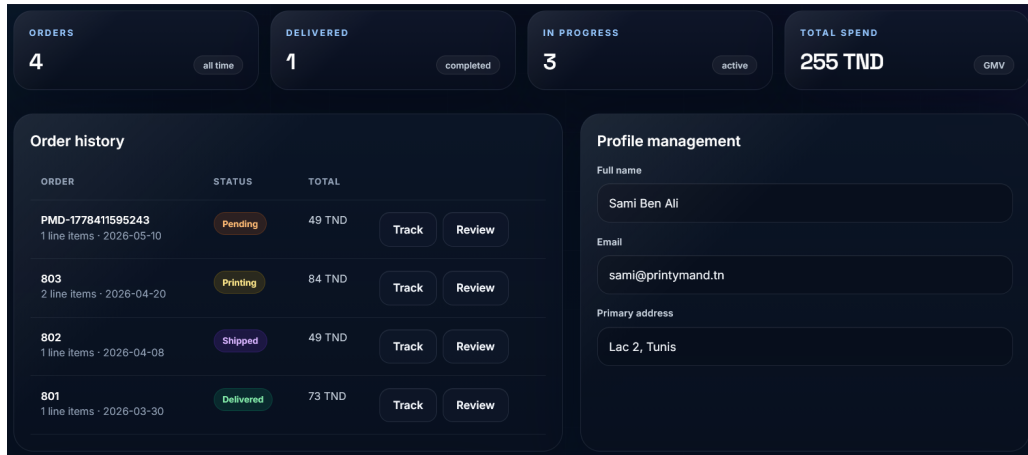


Figure 5.1: Customer dashboard interface screenshot.

5.4.2 Designer dashboard

The designer dashboard supports design management and analytics. Designers can create or update designs, define design status, assign products to artworks, and configure product-specific placement defaults. The dashboard also displays performance indicators such as earnings, number of designs, order count, and conversion.

The design model includes title, image, designer information, category, description, tags, rating, sales, views, engagement rate, conversion rate, price, status, assigned products, product configurations, and timestamps.

5.4.3 Printer dashboard

The printer dashboard allows printing partners to manage products and incoming order lines. Printers can update fulfillment statuses such as pending, accepted, printing, shipped, delivered, and rejected. The dashboard also displays product count, revenue, fulfillment rate, and rating.

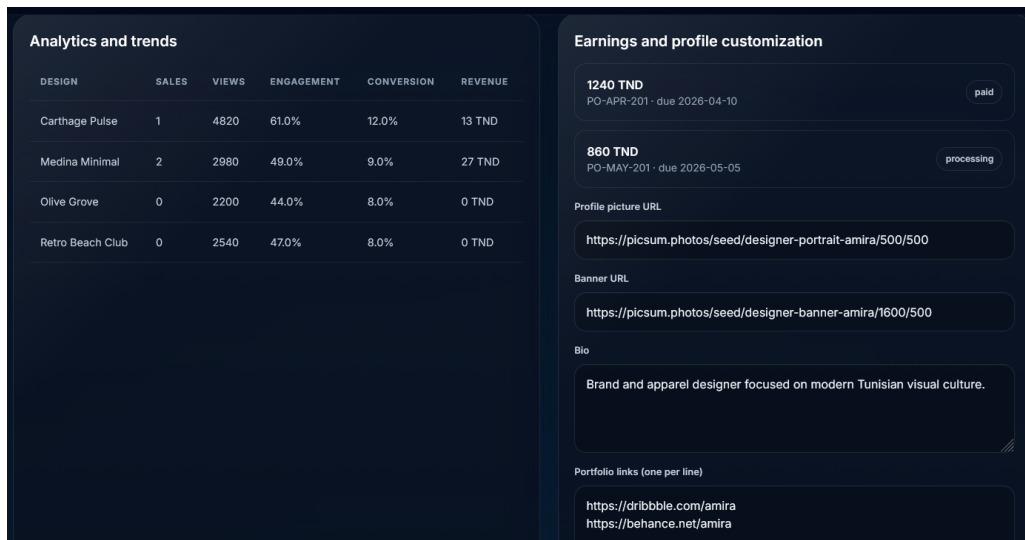


Figure 5.2: Designer dashboard interface screenshot.

The product model includes printer identity, name, category, description, base price, colors, sizes, images, availability, lead time, rating, total orders, and assigned design identifiers.

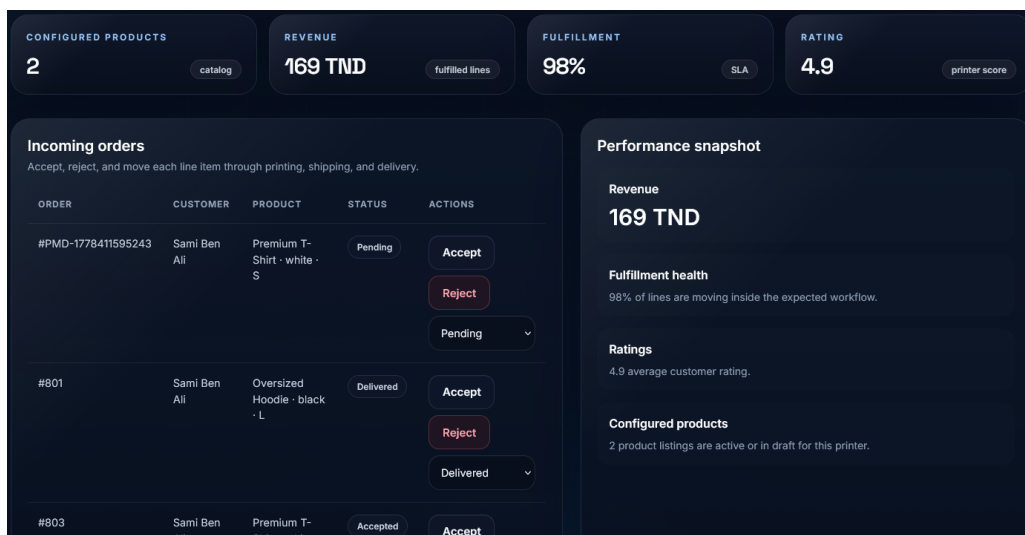


Figure 5.3: Printer dashboard interface screenshot.

5.4.4 Admin dashboard

The admin dashboard provides an overview of users, orders, revenue, active designs, and active printers. It also supports account supervision operations such as role updates, rank updates, and account suspension. This dashboard provides the governance layer required for a marketplace with multiple stakeholders.

For technical monitoring, the health module checks server memory and PostgreSQL connectivity. This gives administrators visibility over essential platform health indicators.

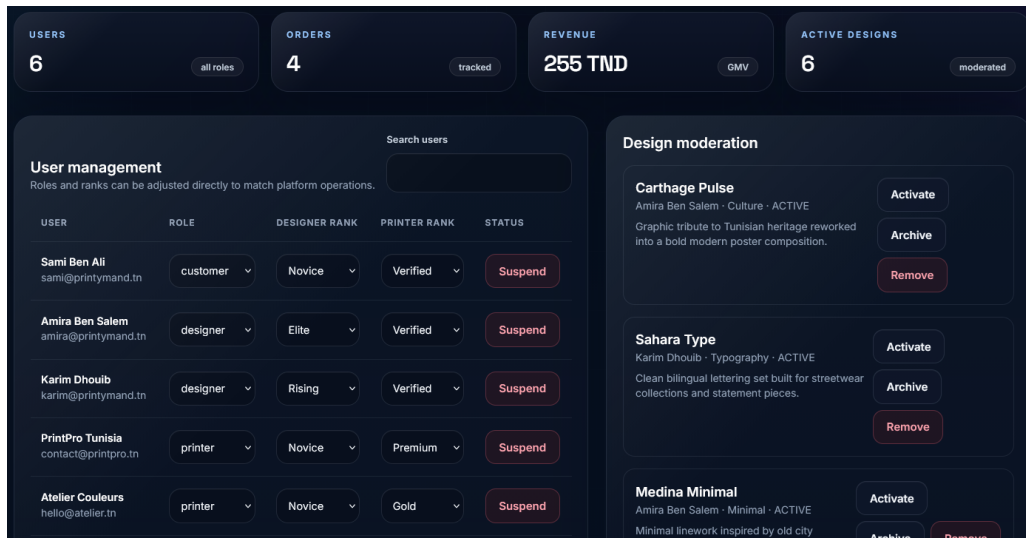


Figure 5.4: Admin dashboard interface screenshot.

5.4.5 Back-end optimization and technical services

We added several technical services to improve reliability and maintainability:

- **Validation:** incoming requests are validated through DTOs and a global validation pipe.
- **Exception handling:** global filters standardize error responses.
- **Logging:** an interceptor records request and response behavior.
- **Throttling:** request-rate limits protect the API from abusive usage.
- **Transactions:** transaction handling preserves database consistency during sensitive operations.
- **Realtime services:** realtime services support notifications, presence, and live updates.

5.4.6 Database consolidation

The database schema supports the main marketplace entities. Users are separated from designer and printer inventories. Designs can belong to albums and tags. Products belong

to printer inventories. Orders contain order items associated with products and designs. This model provides a coherent foundation for the full marketplace workflow.

5.5 Validation and evaluation

At the end of Cycle 3, the platform included the operational spaces required by customers, designers, printers, and administrators. The dashboards validated the multi-role nature of Printymand, while the technical services strengthened security, monitoring, and maintainability. This cycle completed the platform by connecting commercial workflows to governance and administration.

5.6 Conclusion

The third Spiral cycle added governance, role-specific workspaces, administration, monitoring, and optimization to Printymand. It completed the platform by providing each actor with the tools required to participate in the marketplace. The final conclusion summarizes the work achieved and presents perspectives for future enhancement.

General Conclusion

This PFA project allowed us to design and implement Printymand, a print-on-demand marketplace platform that connects customers, designers, printers, and administrators. Through this work, we addressed a realistic software engineering problem involving multi-role workflows, product customization, order management, authentication, authorization, database modeling, and full-stack architecture.

On the front-end side, we developed an Angular application that provides marketplace browsing, authentication, role-specific dashboards, product customization, printer selection, cart management, checkout, order tracking, reviews, and profile management. On the back-end side, we implemented a NestJS API with authentication, authorization, persistence, validation, exception handling, monitoring, and modular business services. PostgreSQL was used to model the main entities of the marketplace and preserve the relationships between users, products, designs, printers, and orders.

The Spiral Model helped us organize the project progressively. The first cycle established authentication, onboarding, and marketplace discovery. The second cycle implemented customization, cart, checkout, and order tracking. The third cycle completed the platform with dashboards, administration, governance, monitoring, and technical optimization.

This project was a valuable academic and technical experience. It strengthened our skills in Angular, NestJS, PostgreSQL, TypeScript, software architecture, and project organization. It also showed the importance of communication between front-end and back-end development when building a coherent platform.

As perspectives, Printymand can be enhanced with advanced recommendation algorithms, additional online payment providers, automated invoice generation, real-time notifications, multilingual deployment, and production cloud hosting. These improvements would extend the platform and prepare it for broader market usage.

In conclusion, Printymand demonstrates how a local print-on-demand marketplace can be

designed and implemented with modern web technologies while answering the needs of customers, designers, printers, and administrators.

Webography

- [1] Angular, *Angular documentation*, <https://angular.dev/>.
- [2] Microsoft, *Typescript documentation*, <https://www.typescriptlang.org/docs/>.
- [3] NestJS, *Nestjs documentation*, <https://docs.nestjs.com/>.
- [4] P. G. D. Group, *Postgresql documentation*, <https://www.postgresql.org/docs/>.
- [5] IETF, *Json web token (jwt)*, <https://datatracker.ietf.org/doc/html/rfc7519>.
- [6] T. Labs, *Tailwind css documentation*, <https://tailwindcss.com/docs>.